

Mobile Test Strategy

This is how I would approach testing the mobile version of this app. The product I am looking at is the website `automationexercise.com`, and for this exercise I am treating it as the reference for a native iOS and Android app that does the same things.

My goal in this document is not to list every test I could possibly run. It is to show what I would focus on first, what I would leave for later, and the reasoning behind those choices. I think that prioritisation is the real skill in QA, so that is what I want to make clear.

What I am trying to do here

When I plan testing for an app like this, I start by asking a simple question: where does this product make or lose money and trust? For an online shop, that is a fairly small set of things. If those break, nothing else matters. So I put most of my early effort there, and I am deliberate about the parts I choose to leave until later.

The user flows I would test first

I prioritise flows by two things together: how important they are to the business, and how likely they are to break specifically on a phone. Here are the five I would start with, and why.

1. **Creating an account and logging in.** This is the front door. If sign up or login does not work, the user never reaches the cart or the checkout, so everything else is blocked. It also has a lot of input fields (email, password, personal details), and it is the screen most affected by mobile keyboards, autofill, and staying logged in between sessions.
2. **Finding products (browsing, search, and filters).** This is the top of the journey. If people cannot find what they want, they cannot buy it. On a phone this is very touch heavy (lots of scrolling and tapping), images need to load nicely even on a weak connection, and it is usually the first place performance problems show up.
3. **Managing the cart (adding, updating, removing, and keeping items).** This is the bridge between looking and buying. The part I care about most on mobile is whether the cart survives. People get interrupted constantly on phones, so I want to know the cart is still there when they come back.
4. **Checkout and payment.** This is the single most important flow for revenue. A problem here is a direct, measurable loss, so it gets the deepest testing even though it gets less traffic than browsing. I would test both the happy path and a lot of "what if the user does something wrong" cases.
5. **Staying logged in and keeping state.** This runs across everything. It covers things like staying signed in, the cart surviving after the app is closed, links opening the right screen, and the back

button behaving as expected. This is the area where a mobile app most often behaves differently from the website.

If I could only pick three of these, I would keep account access, cart, and checkout, because that is the path that actually leads to a sale.

What I would automate and what I would test by hand

The way I split this is simple. I automate the things that are stable and that I will need to check over and over again. I test by hand the things that need human judgement or that depend on a real device.

Things I would automate (my safety net):

- The main account flows: registering, logging in, logging in with the wrong password, and trying to register an email that already exists. These have clear right and wrong answers, so a machine can check them on every build. I have already built these in this project.
- Adding products to the cart and checking that the cart keeps its contents after a refresh. This is also already built.
- A basic checkout check that reaches the payment step with a known product, stopping before any real payment.
- Running the same checks on both iOS and Android, which is cheap to do once the framework is set up and catches differences between the two platforms.

I automate these because they run constantly, they have a clear pass or fail, and re-checking them by hand on every release across two platforms would be slow and error prone.

Things I would test by hand (where judgement matters):

- First time use and permission prompts (notifications, location), including what happens when the user says no.
- Real gestures and how the app feels in the hand: swiping, long pressing, pull to refresh, reaching things one handed, larger font sizes, and dark mode.
- Interruptions and bad connections: a phone call mid checkout, the app being sent to the background, airplane mode, a weak signal, and recovering when the connection comes back.
- Visual polish and accessibility: how the layout looks on different screen shapes, and how it works with screen readers like VoiceOver and TalkBack.

I keep these manual because they are hard to judge automatically, they change a lot, and a person can evaluate them faster and more sensibly than a brittle script could.

The biggest risks that are specific to mobile

These are the things that worry me most about this product running as a real app, rather than as a website.

1. **The app being closed and reopened by the phone.** This is my number one concern, so it is worth explaining in plain terms. On a phone, people switch apps all the time. Someone might add

an item to the cart, then jump to WhatsApp to ask a friend, take a call, or just lock the screen. While the app is sitting in the background, the phone's operating system is allowed to shut it down on its own to free up memory. This happens a lot, especially on cheaper Android phones. When the user comes back, the app has to rebuild itself from scratch. The question I would test is: does the cart still have the items, and is the user still logged in? On a website this almost never happens, because the page just stays open in the browser. On a phone it happens constantly, and that is exactly why something can work perfectly on the web but still break in the app.

2. **Bad or changing connections.** Phones move in and out of signal. The app needs to handle a slow connection, a request that drops halfway, and being fully offline, without confusing the user or, worst of all, charging them twice at checkout.
3. **Different devices and screen sizes.** There are a huge number of Android phones with different sizes, operating system versions, and manufacturer tweaks, plus iPhones with notches and safe areas. Buttons and layouts have to hold up across all of them.
4. **Payment safety.** I want to be sure the user cannot accidentally submit a payment twice on a flaky connection, that switching to a banking app and back works correctly, and that no duplicate orders or charges are created.
5. **Performance on cheaper phones.** Long, image heavy product lists can become slow and jerky on low end devices, and can use a lot of memory and battery. I would keep an eye on scrolling smoothness and how long the app takes to start.
6. **Permissions and links.** Push notifications, and links that should open the right screen inside the app, both need to behave correctly.
7. **Keeping user data safe on the device.** Login tokens and personal details should be handled carefully, and the right thing should happen when the user logs out or loses their phone.

What I would not test at the start

Choosing not to test something is a decision, not an oversight, so I want to be open about it. Early on, I would not spend time on:

- **Testing on every possible device.** I would start with two or three that represent most users (one cheaper Android, one modern Android, one current iPhone) and grow from there. Trying to cover every device from day one is not worth the effort.
- **Pixel perfect visual checks.** Useful later, but early on they create a lot of noise and maintenance for little value.
- **Different languages, currencies, and right to left layouts.** Only worth it if and when the product targets those markets.
- **Formal performance benchmarks as a hard gate.** I would still look at performance by hand early, but strict targets come after the main tests are trustworthy.
- **Rarely used features**, such as minor account settings or secondary pages, until the core buying journey is solid.

- **Checking that ads and analytics are correct**, beyond making sure they do not get in the way of the user. (In fact, the ad scripts on the reference site actively got in the way of automation, so I treat them as noise to work around rather than something to validate.)

The thread running through all of this is the same: protect the buying journey first, then widen the coverage once I am confident the core is solid.

In short

I would test the buying journey first, meaning account access, browsing, cart, and checkout, before anything else. I would automate the repeatable checks and explore the device and human dependent parts by hand. I would treat the app being closed and reopened, bad connections, device variety, and payment safety as the defining mobile risks. And I would hold off on breadth, like full device coverage, visual checks, extra languages, and strict performance targets, until the core is solid, because knowing what to leave for later is part of doing this well.